

Algorithmic Approaches to Hyperparameter Tuning

May 20, 2026

1 Random forest

1.1 Exhaustive Grid Search

Grid Search is a brute-force hyperparameter optimization method. It defines a discrete set of values for each hyperparameter and exhaustively evaluates the model's fitness (validation accuracy) for every possible combination in the Cartesian product of these sets.

Algorithm 1 2D Grid Search

```
1: Input:    Grid boundaries  $n_{min}, n_{max}, depth_{min}, depth_{max}$ , and step counts  $n_{steps}, depth_{steps}$ 
2:  $N_{grid} \leftarrow \text{linspace}(n_{min}, n_{max}, n_{steps})$ 
3:  $D_{grid} \leftarrow \text{linspace}(depth_{min}, depth_{max}, depth_{steps})$ 
4:  $best\_acc \leftarrow -\infty$ 
5:  $best\_params \leftarrow \emptyset$ 
6: for  $n \in N_{grid}$  do
7:   for  $d \in D_{grid}$  do
8:      $score \leftarrow \text{Evaluate}(n, d)$  ▷ Train RF and calculate validation accuracy
9:     if  $score > best\_acc$  then
10:       $best\_acc \leftarrow score$ 
11:       $best\_params \leftarrow \{n, d\}$ 
12:    end if
13:  end for
14: end for
15: return  $best\_params, best\_acc$ 
```

1.2 Genetic Algorithm (GA)

The Genetic Algorithm is an evolutionary metaheuristic inspired by natural selection. The implemented 2D GA maintains a population of candidate solutions (individuals), evaluating their fitness at each generation. It applies Elitism (preserving the best individual), Tournament Selection (size 3) to choose parents, Blend Crossover to generate offspring, and Gaussian Mutation to introduce genetic diversity.

Algorithm 2 Genetic Algorithm for Hyperparameter Tuning

```
1: Input: Population size  $P$ , Generations  $G$ , Mutation rate  $P_m = 0.2$ , Bounds  $B$ 
2: Initialize population  $\mathcal{P}_0 \in \mathbb{R}^{P \times 2}$  uniformly within bounds  $B$ 
3:  $gbest\_score \leftarrow -\infty$ 
4: for  $t = 1 \rightarrow G$  do
5:   Evaluate fitness  $\mathcal{F}$  for each individual  $x_i \in \mathcal{P}_{t-1}$ 
6:   Update global best ( $gbest\_pos, gbest\_score$ ) if a better fitness is found
7:    $\mathcal{P}_t[0] \leftarrow x_{\arg \max(\mathcal{F})}$  ▷ Elitism: Carry over the best individual
8:   for  $i = 1 \rightarrow P - 1$  do
9:      $p_1 \leftarrow \text{TournamentSelection}(\mathcal{P}_{t-1}, \mathcal{F}, \text{size} = 3)$ 
10:     $p_2 \leftarrow \text{TournamentSelection}(\mathcal{P}_{t-1}, \mathcal{F}, \text{size} = 3)$ 
11:     $\beta \leftarrow U(0, 1)^2$  ▷ Blend Crossover
12:     $child \leftarrow p_1 \cdot \beta + p_2 \cdot (1 - \beta)$ 
13:    if  $U(0, 1) < P_m$  then ▷ Gaussian Mutation
14:       $child[0] \leftarrow child[0] + \mathcal{N}(0, 20)$ 
15:       $child[1] \leftarrow child[1] + \mathcal{N}(0, 5)$ 
16:    end if
17:     $\mathcal{P}_t[i] \leftarrow \text{Clip}(child, B)$ 
18:  end for
19: end for
20: return Round( $gbest\_pos$ ),  $gbest\_score$ 
```

1.3 Particle Swarm Optimization (PSO)

PSO is a swarm-intelligence algorithm inspired by the flocking behavior of birds. Each candidate solution is a "particle" traversing the search space. A particle's movement is guided by its own best-known position ($pbest$) and the entire swarm's best-known position ($gbest$). The implemented standard PSO uses inertia weight $w = 0.5$ and acceleration coefficients $c_1 = 1.5, c_2 = 1.5$.

Algorithm 3 Particle Swarm Optimization (2D)

```
1: Input: Particles  $N = 15$ , Iterations  $I = 20$ ,  $w = 0.5$ ,  $c_1 = 1.5$ ,  $c_2 = 1.5$ , Bounds  $B$ 
2: Initialize positions  $X \in \mathbb{R}^{N \times 2}$  uniformly within bounds  $B$ 
3: Initialize velocities  $V \in \mathbb{R}^{N \times 2}$  uniformly in  $[-1, 1]$ 
4:  $Pbest \leftarrow X$ 
5:  $Pbest\_scores \leftarrow -\infty \times \mathbf{1}_N$ 
6:  $Gbest\_pos \leftarrow \emptyset$ ,  $Gbest\_score \leftarrow -\infty$ 
7: for  $t = 1 \rightarrow I$  do
8:   for  $i = 1 \rightarrow N$  do
9:      $score \leftarrow \text{Evaluate}(\text{Round}(X_i))$ 
10:    if  $score > Pbest\_scores[i]$  then
11:       $Pbest\_scores[i] \leftarrow score$ 
12:       $Pbest[i] \leftarrow X_i$ 
13:    end if
14:    if  $score > Gbest\_score$  then
15:       $Gbest\_score \leftarrow score$ 
16:       $Gbest\_pos \leftarrow X_i$ 
17:    end if
18:  end for
19:   $R_1, R_2 \sim U(0, 1)^{N \times 2}$ 
20:   $V \leftarrow w \cdot V + c_1 \cdot R_1 \cdot (Pbest - X) + c_2 \cdot R_2 \cdot (Gbest\_pos - X)$ 
21:   $X \leftarrow \text{Clip}(X + V, B)$ 
22: end for
23: return  $\text{Round}(Gbest\_pos), Gbest\_score$ 
```

1.4 Optuna Optimization (Bayesian / TPE)

Optuna operates fundamentally differently from GA or PSO by treating hyperparameter tuning as a Sequential Model-Based Optimization (SMBO) problem. By default, it utilizes the Tree-structured Parzen Estimator (TPE) algorithm. Rather than random or heuristic-driven jumps, TPE builds a probabilistic surrogate model of the objective function $P(x|y)$ and $P(y)$, actively sampling the hyperparameter combinations that maximize the Expected Improvement (EI).

Algorithm 4 Optuna Optimization Loop

```
1: Input: Number of trials  $T = 30$ , Parameter search spaces  $\mathcal{S}$ 
2: Initialize Trial History  $\mathcal{H} \leftarrow \emptyset$ 
3: for  $t = 1 \rightarrow T$  do
4:    $\theta_t \leftarrow \text{TPE\_Suggest}(\mathcal{H}, \mathcal{S}) \triangleright$  Suggests params maximizing Expected Improvement
5:    $n\_est \leftarrow \theta_t['n\_estimators']$ 
6:    $depth \leftarrow \theta_t['max\_depth']$ 
7:    $min\_split \leftarrow \theta_t['min\_samples\_split']$ 
8:    $score_t \leftarrow \text{Evaluate}(n\_est, depth, min\_split)$ 
9:    $\mathcal{H} \leftarrow \mathcal{H} \cup \{(\theta_t, score_t)\} \triangleright$  Update the surrogate model with new observation
10: end for
11: return  $\theta_{best} \in \mathcal{H}$  corresponding to  $\max(score)$ 
```
